

Day 10 — Tree-Based Models for Structured Finance Data

Beginner-friendly summary

Tree models are often among the strongest choices for **structured/tabular enterprise data**: transactions, vendors, departments, amounts, risk scores, approval delays, budgets, account attributes, and other mixed business features.

The core progression is:

```
Decision Tree
  ↓
Easy to understand
But high variance / easy to overfit

Random Forest
  ↓
Many independent-ish trees
Average them
Reduce variance

Gradient Boosting
  ↓
Build trees sequentially
Each tree fixes previous errors
Usually stronger predictive performance

XGBoost / LightGBM / CatBoost
  ↓
Highly optimized implementations
Regularization + missing-value handling
+ efficient training + categorical strategies
```

The senior-level lesson is not:

| "Gradient boosting is always best."

It is:

| **Pick the model based on data structure, validation design, business loss, latency, interpretability, temporal stability, and operational constraints.**

For finance data, an excellent model evaluated with the wrong temporal split can be considerably more dangerous than a simpler model evaluated correctly.

1. Concise model comparison

Model	Main strength	Main weakness	Choose when
Decision Tree	Simple, interpretable, low latency	High variance, overfits easily	Rules/explainability matter strongly

Model	Main strength	Main weakness	Choose when
Random Forest	Robust, stable, little tuning	Larger memory, many trees	Strong general-purpose tabular baseline
Gradient Boosting	Excellent nonlinear tabular performance	More sensitive to tuning	Predictive performance is important
XGBoost	Strong regularization and mature ecosystem	More hyperparameters	General structured ML problems
LightGBM	Very fast on large datasets	Can overfit small datasets	Very large/wide tabular datasets
CatBoost	Excellent categorical handling	Can be computationally heavier	Many/high-cardinality categorical variables
Neural Network	Representation learning	Often unnecessary for ordinary tabular data	Huge data or multimodal/unstructured inputs
LLM	Text/reasoning interface	Poor default choice for numeric tabular prediction	Natural language/document reasoning

A sensible tabular sequence is often:

```

Simple baseline
  ↓
Decision Tree
  ↓
Random Forest
  ↓
Gradient Boosting
  ↓
XGBoost / LightGBM / CatBoost
  ↓
Complexity justified?

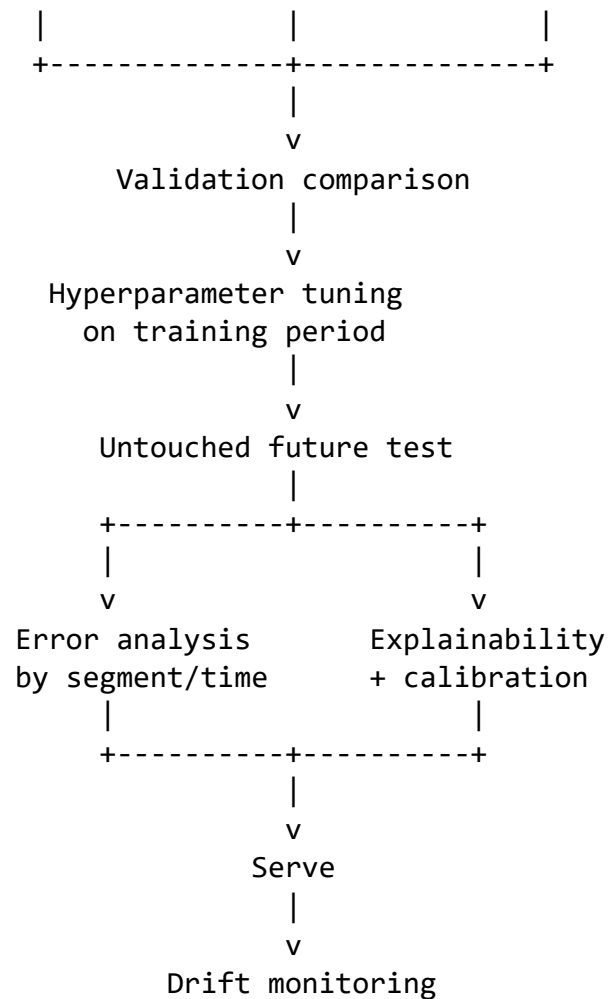
```

2. Production ML workflow

```

Raw finance data
  |
  v
Point-in-time correct features
  |
  v
Temporal train / validation / test
  |
  +-----+-----+
  |         |         |
  v         v         v
Decision Tree Random Forest Gradient Boosting

```



The most important box for finance is often not the model.

It is:

point-in-time correct temporal validation.

3. Decision trees

3.1 What a decision tree actually learns

Suppose we predict whether an expense requires investigation.

Features might include:

```

amount
vendor_risk
budget_variance
approval_delay
department
region
vendor_tenure
  
```

A tree might learn:

```

vendor_risk > 0.45?
  /      \
 No      Yes
  |      |
budget_var > .30?  amount > 5000?
  /      \      /      \
low      high medium    high
risk     risk risk      risk

```

It recursively divides feature space into regions.

Each final region is a **leaf**.

3.2 How does a tree choose a split?

The tree considers candidate conditions such as:

```

amount <= 5000
amount <= 10000
vendor_risk <= 0.3
vendor_risk <= 0.5
approval_delay <= 7

```

It selects the split producing the largest reduction in impurity.

For classification, common measures are **Gini impurity** and entropy.

Gini impurity

$$Gini = 1 - \sum_k p_k^2$$

For binary classification:

```

50% positive / 50% negative
→ high impurity

100% positive
→ impurity = 0

```

A good split produces purer child nodes.

The weighted impurity after splitting is roughly:

$$I_{split} = \frac{n_L}{n} I_L + \frac{n_R}{n} I_R$$

We want maximum:

$$I_{parent} - I_{split}$$

Regression trees

For regression, such as predicting next-month spend, the tree commonly minimizes squared error.

Suppose a leaf contains:

```
100
105
110
98
103
```

The prediction is approximately their mean.

A split is useful when separating the observations reduces within-leaf variance.

4. Depth, leaves, and overfitting

Imagine a tree with unlimited depth.

Eventually it might produce rules like:

```
vendor_id = X
AND
amount = 7132.41
AND
transaction_day = Tuesday
AND
approval_delay = 4.3

→ positive
```

That may perfectly explain training data.

It probably does not represent a stable business relationship.

Deep tree

```
Low bias
High variance
```

Shallow tree

```
Higher bias
Lower variance
```

Important controls include:

```
max_depth
min_samples_split
```

```
min_samples_leaf
max_leaf_nodes
min_impurity_decrease
```

For business datasets, `min_samples_leaf` can be particularly useful because it prevents decisions based on tiny populations.

5. Pruning

There are two broad approaches.

Pre-pruning

Stop the tree from becoming too complicated.

For example:

```
max_depth=5
min_samples_leaf=100
```

Post-pruning

Grow a larger tree and remove branches that provide insufficient improvement.

Cost-complexity pruning can conceptually be represented as:

$$R_{\alpha}(T) = R(T) + \alpha |Leaves(T)|$$

where:

- $R(T)$ = prediction error
- $|Leaves(T)|$ = number of leaves
- (α) = penalty on complexity

Large alpha:

```
more pruning
→ smaller tree
→ potentially lower variance
```

6. Why decision trees overfit

Trees repeatedly search many possible splits.

Eventually they can find patterns caused only by random variation.

Example:

```
Training data:

Region=APAC
VendorType=B
```

Amount > ₹73,421

Day = Thursday

appears strongly associated with exceptions

Perhaps there were only eight such observations.

The relationship may disappear next month.

That's why tree constraints and validation matter.

7. Random forests

Random forests attack the main weakness of decision trees:

variance.

Instead of trusting one tree:

```
Dataset
|
+--> Tree 1
+--> Tree 2
+--> Tree 3
+--> ...
+--> Tree N
      |
      v
Average / Vote
```

8. Bagging

Bagging means:

Bootstrap Aggregating

Each tree receives a bootstrap sample.

If the original dataset contains N observations, draw N observations **with replacement**.

Consequently:

```
some observations appear multiple times
some observations do not appear
```

Train different trees on different samples.

Then average them.

Regression:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M T_m(x)$$

Classification generally averages predicted probabilities or votes.

9. Why averaging helps

Suppose individual trees are noisy.

```
Tree 1 → slightly too high
Tree 2 → slightly too low
Tree 3 → different error
Tree 4 → different error
```

If their errors aren't perfectly correlated, averaging reduces variance.

But there's another problem.

If every tree always chooses the strongest feature first, the trees can become highly correlated.

Random forests therefore introduce another form of randomness.

10. Feature subsampling

At each split, consider only a subset of features.

Instead of:

```
20 available features
→ tree considers all 20
```

perhaps:

```
20 available features
→ randomly select ~sqrt(20)
→ search split among those
```

This encourages tree diversity.

The intuition is:

```
Bootstrap samples
+
Feature subsampling
↓
Less correlated trees
```


↓
Averaging becomes more effective

11. Out-of-bag evaluation

Because bootstrap sampling leaves some observations unused for a particular tree, those observations are called **out-of-bag observations**.

Example:

```
Tree 17 training bootstrap
|
+-- rows used → training
|
+-- rows absent → OOB
```

Those unused rows can provide an internal estimate of generalization.

This is convenient for IID datasets.

Critical finance caveat

OOB evaluation **does not replace temporal validation**.

If your transactions are:

```
2023
2024
2025
2026
```

random bootstrap samples mix periods.

That can tell you something about random-sample performance but not:

| Can a model trained yesterday predict tomorrow?

For changing finance systems, future-period evaluation matters more.

12. Random forest strengths and weaknesses

Strengths

- nonlinear relationships
- automatic interaction discovery
- robust baseline
- little feature scaling required
- relatively insensitive to individual noisy observations
- parallelizable training
- usually less overfit than individual trees

Weaknesses

- large models
 - potentially expensive inference with hundreds/thousands of trees
 - weaker extrapolation
 - less transparent than one tree
 - feature importance can mislead
 - probability estimates may require calibration
 - sometimes weaker than boosting on structured datasets
-

13. Gradient boosting

Random forest asks:

| What if we train many different trees and average them?

Gradient boosting asks:

| What if every new tree focuses on what the previous trees still get wrong?

Conceptually:

```
Initial prediction
  |
  v
Calculate error
  |
  v
Train small Tree 1
  |
  v
Improve prediction
  |
  v
Calculate remaining error
  |
  v
Train Tree 2
  |
  v
Improve again
  |
  ...
```

The trees are sequential rather than independent.

14. Residual fitting intuition

For squared-error regression:

```
actual = 100  
current prediction = 70  
residual = +30
```

Another observation:

```
actual = 40  
current prediction = 55  
residual = -15
```

The next tree attempts to model those residual patterns.

For squared-error loss:

$$r_i = y_i - F(x_i)$$

Then:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

where:

- (F_m) = new ensemble
- (h_m) = new tree
- (η) = learning rate

For other losses, boosting fits the **negative gradient of the loss**, which generalizes the residual idea.

15. Learning rate versus number of trees

This relationship is fundamental.

Large learning rate

```
Each tree makes a large correction  
→ fewer trees  
→ faster  
→ greater risk of overfitting/instability
```

Small learning rate

```
Each tree makes a small correction  
→ more trees required  
→ often smoother generalization  
→ greater training/inference cost
```

Typical tuning relationship:

```
smaller learning_rate
      ↓
larger n_estimators
```

Don't tune them independently without considering this interaction.

16. Gradient boosting regularization

Important controls include:

```
learning_rate
n_estimators
max_depth / number of leaves
min_samples_leaf
subsample
feature sampling
L1 penalties
L2 penalties
early stopping
```

One powerful principle is using **small trees**.

For example:

```
depth = 1
```

captures single-feature effects.

```
depth = 2
```

can represent two-way interactions.

```
depth = 3+
```

captures increasingly complex interactions but increases capacity.

17. Bagging versus boosting

This distinction is interview-important.

Random forest

```
Tree 1 ┌
Tree 2 ┤
Tree 3 ┤ ──> average
Tree 4 ┤
Tree 5 └
```

Independent-ish
Primarily variance reduction

Gradient boosting

Tree 1
↓ fixes remaining errors
Tree 2
↓
Tree 3
↓
Tree 4

Sequential
Primarily reduces bias while regularization controls variance

That's simplified, but it is the correct intuition.

18. XGBoost vs LightGBM vs CatBoost

All three are gradient-boosted tree frameworks, but their engineering and algorithmic choices differ.

XGBoost

XGBoost is an extremely mature general-purpose option.

Important ideas include:

- first- and second-order loss information
- gradient and Hessian calculations
- explicit regularization
- column subsampling
- row subsampling
- sparse-aware processing
- histogram-based algorithms
- learned default directions for missing values
- strong ecosystem/tooling

Conceptually, tree construction considers not just:

How much does this split improve training error?

but also:

Is the improvement large enough to justify adding complexity?

Good default when:

I have structured data and want a mature, high-performance boosting implementation.

19. LightGBM

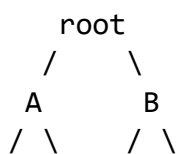
LightGBM was designed strongly around efficiency and scale.

Key concepts include:

- histogram-based split finding
- leaf-wise tree growth
- efficient large-data training
- categorical feature support
- Gradient-based One-Side Sampling concepts
- Exclusive Feature Bundling concepts

A major difference is tree growth.

Traditional depth-wise growth:



LightGBM commonly uses leaf-wise growth:

```
Find leaf offering  
largest loss reduction  
↓  
split that leaf  
↓  
repeat
```

This can converge quickly.

But aggressive leaf-wise growth can also overfit smaller datasets unless constrained.

Good when:

| Dataset size, width, and training efficiency are major concerns.

20. CatBoost

CatBoost is particularly attractive when categorical variables matter.

Finance datasets often contain:

```
merchant  
vendor  
cost_center  
department  
country  
payment_type
```

```
product
account_category
```

Naive target encoding can introduce leakage.

For example:

```
vendor → historical fraud rate
```

If an observation's own target contributes to the encoding used to predict that observation, leakage occurs.

CatBoost uses **ordered target-statistics concepts** designed to reduce that problem.

It also uses ordered boosting techniques and commonly symmetric/oblivious tree structures.

Good when:

Categorical variables are numerous or high-cardinality and encoding them safely is inconvenient.

21. Practical comparison

Dimension	XGBoost	LightGBM	CatBoost
General tabular performance	Excellent	Excellent	Excellent
Very large datasets	Strong	Often excellent	Strong
High-cardinality categoricals	Requires care	Good native support	Major strength
Training speed	Strong	Often very fast	Depends on dataset
Overfit risk on small data	Manageable	Leaf-wise growth needs care	Manageable
Missing values	Native support	Native support	Native support
Ecosystem maturity	Excellent	Excellent	Excellent
Tuning complexity	Moderate/high	Moderate/high	Often strong defaults

The difference is rarely:

Framework A is universally better.

The correct question is:

Which framework best matches my dataset and operational constraints?

22. Why trees work so well on nonlinear finance data

Suppose fraud risk behaves like:

```
amount < 10,000
→ normal

amount > 10,000 AND new_vendor
→ elevated

amount > 10,000 AND old_vendor
→ normal

amount > 50,000 AND unusual_region
→ very elevated
```

A linear model struggles unless you manually construct interactions.

A tree naturally represents them.

23. Interactions

Imagine:

```
large transaction alone
→ not necessarily suspicious

new vendor alone
→ not necessarily suspicious

large transaction + new vendor
→ suspicious
```

A tree can learn:

```
new_vendor?
|
Yes
|
amount > 50k?
|
Yes
|
High risk
```

No explicit interaction feature is required.

24. Feature scaling

Linear and neural models can care strongly about scale.

For trees:

```
amount = 1,000,000  
risk_score = 0.73  
days = 7
```

is normally fine.

Trees care about ordering and split thresholds rather than Euclidean scale.

You generally do **not** need:

```
StandardScaler  
MinMaxScaler
```

solely for decision trees.

25. Missing values

Missingness deserves domain reasoning.

Suppose:

```
approval_date = missing
```

That could mean:

1. data quality failure,
2. approval has not occurred,
3. approval isn't required,
4. upstream schema changed.

Those are completely different meanings.

Some boosting implementations can learn default directions for missing observations.

But native support does **not** remove the need to understand why values are missing.

Sometimes a useful feature is:

```
approval_date_missing = 1
```

because missingness itself contains information.

That can also become dangerous if it represents an unstable upstream operational artifact.

26. Categorical variables

You have several choices.

One-hot encoding

Suitable for relatively low cardinality:

```
region:  
India  
US  
UK  
Germany
```

becomes:

```
region_india  
region_us  
region_uk  
region_germany
```

Target/statistical encoding

Useful for high cardinality but carries leakage risk.

Native categorical support

LightGBM and CatBoost provide useful capabilities here.

High-cardinality warning

A feature such as:

```
customer_id  
transaction_id  
invoice_id
```

can make trees memorize entities.

High apparent validation performance may simply be identity leakage.

27. Hyperparameter tuning

Do not start by searching hundreds of hyperparameters.

Tune model capacity first.

Decision tree

Focus on:

```
max_depth  
min_samples_leaf  
max_leaf_nodes  
ccp_alpha
```

Random forest

Focus on:

```
n_estimators  
max_depth  
min_samples_leaf  
max_features
```

Gradient boosting

Focus on:

```
learning_rate  
n_estimators  
tree depth / leaves  
min samples per leaf  
subsample  
column sampling  
regularization
```

28. Coarse-to-fine tuning

Better:

```
Stage 1  
Broad search  
  ↓  
Find approximate region  
  ↓  
Stage 2  
Narrow search  
  ↓  
Validate stability
```

than:

```
Try 10,000 configurations  
on one validation set  
and select maximum score
```

Why?

Because repeated optimization against a validation set eventually overfits the validation set itself.

29. Validation overfitting

Suppose you try:

```
Model 1  
Model 2  
...  
Model 5000
```

and always examine validation PR-AUC.

Eventually:

| you are indirectly training against the validation dataset.

Your reported validation score becomes optimistic.

Mitigations:

- reserve an untouched test period
- use rolling/temporal cross-validation
- restrict search space
- use nested validation where warranted
- select hyperparameters based on stability, not merely maximum score
- avoid repeatedly checking the final test set
- predefine your main business metric

30. Feature importance

There is no universally correct single feature-importance method.

Gain-based importance

Tree models can track how much reduction in loss/impurity resulted from splits involving each feature.

Conceptually:

```
vendor_risk  
→ responsible for many useful splits  
→ high gain importance
```

Useful for:

- fast debugging
- model inspection
- rough global ranking

But it can mislead.

Features with many possible split opportunities can receive excessive importance.

Correlated variables can also distort the ranking.

31. Permutation importance

Procedure:

1. Measure validation score.
2. Randomly shuffle one feature.
3. Predict again.
4. Measure score degradation.

If:

PR-AUC drops substantially

then the model depended strongly on the feature.

This measures dependence of model performance on that feature.

But it also has weaknesses.

32. Correlated features problem

Suppose:

annual_salary
monthly_salary

contain nearly the same information.

If you permute `annual_salary`, the model may simply use `monthly_salary`.

So permutation importance says:

`annual_salary` not important

That conclusion would be misleading.

Both variables together may be important.

Permutation can also create unrealistic combinations of correlated variables.

33. Explainability hierarchy

Think about explainability in levels.

Level 1 — global

What generally drives the model?

```
feature importance
permutation importance
partial dependence
```

Level 2 — individual prediction

Why did transaction X receive this score?

Methods such as TreeSHAP can help.

Level 3 — business explanation

This is the actual requirement:

```
Which evidence changed the score?
Is it actionable?
Is it stable?
Would the same reasoning apply tomorrow?
```

A mathematically valid attribution isn't automatically a valid causal explanation.

34. When trees beat neural networks

For ordinary structured data such as:

```
50 numeric variables
20 categorical variables
500,000 rows
```

tree boosting is usually a very serious baseline and frequently difficult to beat.

Reasons include:

- little preprocessing
- nonlinear interactions
- relatively strong small/medium-data performance
- robust mixed feature handling
- efficient CPU inference
- mature interpretability tooling

A neural network becomes more compelling when you have:

```
images
text
audio
very large representation-learning problems
embeddings
high-dimensional sequences
multimodal input
```

35. When trees beat LLMs

Consider:

| Predict whether an expense transaction will become delinquent within 60 days.

Available features:

```
amount
vendor tenure
payment history
department
country
risk score
approval duration
historical delinquencies
```

This is primarily a supervised tabular prediction problem.

Using an LLM is usually unnecessary.

A boosted tree offers:

```
lower inference cost
lower latency
deterministic structure
better tabular inductive bias
easier offline evaluation
easier calibration
```

Now suppose you also have:

```
invoice text
contract clauses
analyst notes
email correspondence
```

Then you may combine systems:

```
structured attributes → tree model

documents/text → NLP/LLM

outputs → controlled decision system
```

Don't use an LLM merely because the problem contains the word AI.

36. Serving trade-offs

Model	Latency	Memory	Explainability	Parallelism
Small tree	Excellent	Tiny	Excellent	N/A
Random forest	Moderate	Potentially large	Moderate	Trees parallelizable
Boosted trees	Moderate	Moderate	Moderate	Trees usually sequential in dependency
Large NN	Depends strongly	Often high	Harder	Accelerator-friendly
LLM	Much higher	Very high	Hard	GPU/service dependent

Single tree

Could require only several comparisons:

```
risk > .5?  
amount > 50000?  
vendor_age < 30?
```

Very cheap.

Random forest

Perhaps:

```
500 trees × several comparisons
```

More memory and CPU.

Trees can often be evaluated concurrently.

Gradient boosting

The prediction accumulates tree outputs:

$$F(x) = F_0 + \eta T_1(x) + \dots + \eta T_M(x)$$

More trees therefore generally increase inference cost.

37. Finance-specific temporal failure modes

This is one of the most important Day 10 sections.

Tree models are extremely good at finding patterns.

Unfortunately they are equally capable of finding **temporary operational patterns**.

Failure 1 — random split instead of temporal split

Bad:

```
2024 }  
2025 } — random shuffle —> train/validation/test  
2026 }
```

You measure interpolation across history.

Production requires:

```
past → future
```

Better:

Train	Validation	Test
2024-25	Jan-Mar 2026	Apr-Jun 2026

38. Failure 2 — regime change

Suppose the learned pattern is:

```
high interest rate  
+  
certain borrower profile  
→ default risk
```

Then monetary conditions change materially.

Conditional relationships can move.

This is **concept drift**.

Excellent historical performance may no longer hold.

39. Failure 3 — policy changes

Imagine:

```
Before June:  
transactions > ₹500k manually reviewed  
  
After June:  
threshold changed to ₹250k
```

The label-generating mechanism changes.

Your model may learn historical reviewer policy rather than underlying financial risk.

40. Failure 4 — feedback loops

Suppose the model flags certain vendors.

Investigators examine them more closely.

Therefore more issues are discovered for those vendors.

The future dataset becomes:

```
model prediction
  ↓
investigation behavior
  ↓
observed label
  ↓
future training data
```

Now labels are partly influenced by your own model.

41. Failure 5 — label delay

Suppose default is only known after 90 days.

A transaction from July cannot have a final default label in August.

If you include immature labels:

```
recent examples look artificially negative
```

That's label censoring.

Training datasets must respect label maturity.

42. Failure 6 — feature availability leakage

Your database may contain:

```
final_approval_status
```

today.

But at prediction time, perhaps that field was not yet available.

Historical queries can accidentally reconstruct the present rather than the past.

You need:

| **point-in-time correctness.**

Ask for every feature:

Was this information actually available at the instant the prediction would have been made?

43. Failure 7 — missingness drift

Training:

```
missing vendor score  
→ risky new vendor
```

Production:

```
missing vendor score  
→ scoring API outage
```

Same missing value.

Completely different meaning.

The model can fail dramatically.

44. Failure 8 — category drift

Training:

```
vendor_type =  
A  
B  
C
```

Production introduces:

```
D  
E  
F
```

Or organizational restructures rename departments.

You need robust unknown-category handling and monitoring.

45. Practical task

We'll compare:

1. Decision Tree
2. Random Forest
3. Gradient Boosting

using a **synthetic finance dataset**.

The synthetic target deliberately contains:

- nonlinear effects
- interactions
- missing values
- categorical variables
- temporal drift

That makes it useful for understanding why validation design matters.

46. Reasoning framework before implementation

The important decisions are:

Decision 1 — don't randomly split

Because finance data changes through time:

```
months 1-16 → train  
months 17-20 → validation  
months 21-24 → test
```

Decision 2 — use identical preprocessing

Otherwise differences may come from preprocessing rather than models.

Decision 3 — include nonlinear signal

Otherwise linear/simple models could trivially solve the synthetic problem and teach us little about tree behavior.

Decision 4 — create a future shift

We'll modify risk behavior after month 18.

That lets us test whether:

```
validation winner  
=  
future winner
```

Decision 5 — evaluate multiple metrics

We use:

```
ROC-AUC  
PR-AUC  
log loss
```

But production selection should ultimately use the metric tied to business decision cost.

47. Pseudocode

```
generate finance observations

for each observation:
    generate:
        month
        department
        region
        amount
        budget variance
        approval delay
        vendor tenure
        vendor risk

construct target from nonlinear rules:
    high vendor risk
    large budget variance
    amount × approval delay interaction
    department × region interaction

introduce temporal drift after month 18

introduce some missing values

split:
    months ≤ 16          → training
    months 17..20        → validation
    months ≥ 21          → test

preprocess:
    numeric → median imputation
    categorical → missing-value handling + one-hot encoding

train:
    bounded decision tree
    random forest
    gradient boosting

for each model:
    score validation
    score future test

compare:
    ROC-AUC
    PR-AUC
    log loss

inspect:
    validation-test degradation
    segment errors
```

feature importance

```
select winner only if:  
    metric is business relevant  
    future performance stable  
    important segments acceptable  
    model complexity justified
```

48. Python implementation

```
import numpy as np  
import pandas as pd  
  
from sklearn.base import clone  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import OneHotEncoder  
from sklearn.impute import SimpleImputer  
  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import (  
    RandomForestClassifier,  
    GradientBoostingClassifier,  
)  
  
from sklearn.metrics import (  
    roc_auc_score,  
    average_precision_score,  
    log_loss,  
)  
  
# -----  
# 1. Synthetic finance data  
# -----  
  
rng = np.random.default_rng(42)  
  
n = 12_000  
  
month = rng.integers(1, 25, size=n)  
  
department = rng.choice(  
    ["Finance", "Sales", "Engineering", "Operations"],  
    size=n,  
    p=[0.20, 0.25, 0.30, 0.25],  
)  
  
region = rng.choice(  
    ["India", "US", "EU", "APAC"],
```

```

        size=n,
        p=[0.35, 0.25, 0.20, 0.20],
    )

    vendor_tenure_days = rng.gamma(
        shape=3,
        scale=180,
        size=n,
    )

    amount = np.exp(
        rng.normal(8.2, 1.0, size=n)
    )

    budget_variance_pct = rng.normal(
        0,
        0.18,
        size=n,
    )

    approval_delay_days = rng.gamma(
        shape=2,
        scale=2.5,
        size=n,
    )

    vendor_risk_score = rng.beta(
        2,
        6,
        size=n,
    )

# -----
# 2. Nonlinear target
# -----

late_year = (month % 12 >= 10).astype(int)

logit = (
    -3.4

    # Threshold/nonlinear effect
    + 3.0 * (vendor_risk_score > 0.45)

    # Threshold effect
    + 1.5 * (budget_variance_pct > 0.25)

    # Interaction
    + 1.2 * (
        (amount > 6000)

```

```

        & (approval_delay_days > 5)
    )

    # Categorical interaction
    + 0.9 * (
        (department == "Operations")
        & (region == "APAC")
    )

    # Seasonal effect
    + 0.5 * late_year

    # Temporal drift
    + 0.8 * (
        (month > 18)
        & (region == "EU")
    )

    # Protective nonlinear effect
    - 0.5 * (vendor_tenure_days > 900)
)

probability = 1 / (1 + np.exp(-logit))

target = rng.binomial(
    1,
    probability,
)

df = pd.DataFrame(
    {
        "month": month,
        "department": department,
        "region": region,
        "vendor_tenure_days": vendor_tenure_days,
        "amount": amount,
        "budget_variance_pct": budget_variance_pct,
        "approval_delay_days": approval_delay_days,
        "vendor_risk_score": vendor_risk_score,
        "target": target,
    }
)

# -----
# 3. Introduce missing values
# -----

for column in [
    "vendor_tenure_days",

```



```

    "approval_delay_days",
    "vendor_risk_score",
]:
    missing_rows = rng.choice(
        n,
        size=int(0.03 * n),
        replace=False,
    )

    df.loc[missing_rows, column] = np.nan

# -----
# 4. Temporal split
# -----

train = df[df["month"] <= 16].copy()

validation = df[
    (df["month"] >= 17)
    & (df["month"] <= 20)
].copy()

test = df[df["month"] >= 21].copy()

feature_columns = [
    column
    for column in df.columns
    if column != "target"
]

categorical_columns = [
    "department",
    "region",
]

numeric_columns = [
    column
    for column in feature_columns
    if column not in categorical_columns
]

# -----
# 5. Common preprocessing
# -----

numeric_pipeline = Pipeline(
    steps=[
        (

```

```

        "imputer",
        SimpleImputer(strategy="median"),
    ),
]
)

categorical_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(
                strategy="most_frequent"
            ),
        ),
        (
            "onehot",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False,
            ),
        ),
    ],
)

preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            numeric_pipeline,
            numeric_columns,
        ),
        (
            "categorical",
            categorical_pipeline,
            categorical_columns,
        ),
    ],
)

# -----
# 6. Candidate models
# -----

models = {
    "Decision Tree": DecisionTreeClassifier(
        max_depth=5,
        min_samples_leaf=50,
        random_state=42,
    ),

```

```

    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=400,
        min_samples_leaf=20,
        max_features="sqrt",
        class_weight="balanced_subsample",
        n_jobs=-1,
        random_state=42,
    ),

    "Gradient Boosting":
        GradientBoostingClassifier(
            n_estimators=250,
            learning_rate=0.04,
            max_depth=2,
            min_samples_leaf=30,
            subsample=0.85,
            random_state=42,
        ),
}

# -----
# 7. Evaluation function
# -----

def evaluate(model, data):
    probabilities = model.predict_proba(
        data[feature_columns]
    )[:, 1]

    return {
        "roc_auc": roc_auc_score(
            data["target"],
            probabilities,
        ),

        "pr_auc": average_precision_score(
            data["target"],
            probabilities,
        ),

        "log_loss": log_loss(
            data["target"],
            probabilities,
        ),
    }

# -----

```

```

# 8. Train and compare
# -----

results = []
trained_models = {}

for model_name, estimator in models.items():

    pipeline = Pipeline(
        steps=[
            (
                "preprocessing",
                clone(preprocessor),
            ),
            (
                "model",
                estimator,
            ),
        ]
    )

    pipeline.fit(
        train[feature_columns],
        train["target"],
    )

    trained_models[model_name] = pipeline

    for split_name, dataset in [
        ("validation", validation),
        ("test", test),
    ]:

        metrics = evaluate(
            pipeline,
            dataset,
        )

        results.append(
            {
                "model": model_name,
                "split": split_name,
                **metrics,
            }
        )

results_df = pd.DataFrame(results)

print(
    results_df.sort_values(

```

```
["split", "pr_auc"],  
ascending=[True, False],  
)  
)
```

49. What happens in this reproducible synthetic example?

Using the fixed seed above, my run produced approximately:

Model	Split	ROC-AUC	PR-AUC	Log loss
Decision Tree	Validation	0.797	0.474	0.242
Random Forest	Validation	0.801	0.497	0.370
Gradient Boosting	Validation	0.809	0.527	0.237
Decision Tree	Future test	0.732	0.400	0.345
Random Forest	Future test	0.757	0.436	0.408
Gradient Boosting	Future test	0.755	0.453	0.330

These are **synthetic experiment metrics only**, not project or production claims.

Notice something more important than the absolute numbers:

```
All three models deteriorate  
from validation → later test
```

That's deliberate.

The synthetic data contains temporal drift.

That is exactly the kind of result you should investigate rather than simply saying:

```
| Gradient boosting scored highest, deploy it.
```

50. Why gradient boosting wins this particular experiment

If our primary objective were PR-AUC, gradient boosting is the winner in this synthetic run.

There are good structural reasons.

The target contains:

Threshold effects

```
vendor_risk > 0.45
```

Trees capture these naturally.

Nonlinearity

```
vendor tenure > 900
```

Again natural for trees.

Interaction effects

```
amount > 6000  
AND  
approval_delay > 5
```

A shallow boosted tree can capture such interactions.

Several weak signals

Boosting sequentially improves residual mistakes rather than averaging independently built models.

Therefore it can combine weak rules efficiently.

51. Why doesn't the single tree win?

The underlying function contains multiple relationships.

A restricted tree:

```
max_depth=5
```

has limited capacity.

That's intentional.

If we allow unlimited depth, training performance may increase dramatically while future performance deteriorates.

A single tree therefore faces the strongest:

```
bias ↔ variance
```

trade-off.

52. Why doesn't the random forest clearly dominate?

Random forests reduce variance very effectively.

But the problem contains several structured signals that boosting can sequentially refine.

Random forests build trees independently.

Boosting says:

Tree 1:
capture strongest structure

Tree 2:
focus remaining errors

Tree 3:
focus remaining errors

...

For many structured prediction problems, that refinement gives boosting an advantage.

But you should never state:

| Boosting always beats random forests.

Here, for example, the random forest has slightly higher synthetic future **ROC-AUC**, while gradient boosting has higher **PR-AUC** and better log loss.

That immediately leads back to Day 9:

| **Which metric corresponds to the business decision?**

53. What could invalidate the conclusion?

This is the senior-level part of the practical task.

1. Wrong business metric

Suppose PR-AUC is higher but the business cares about:

cost of false negatives
at operating threshold = 0.73

The ranking could change.

2. Threshold behavior

Model A may have higher global AUC.

Model B may perform better around the actual decision region.

For example:

manual-review capacity = top 2% of transactions

Then top-k precision/recall may matter more than overall ROC-AUC.

3. Temporal instability

We intentionally introduced drift.

Validation:

months 17–20

Test:

months 21–24

All models decline.

That means model family selection alone hasn't solved the actual problem.

4. Hyperparameter tuning

The parameters above are reasonable educational settings, not proof of optimality.

Tuning may alter rankings.

5. Segment performance

Overall gradient boosting could win while failing badly for:

India
Finance department
new vendors
high-value transactions

You need segment analysis.

6. Calibration

Two models can have similar ranking performance but very different probability quality.

If:

predicted risk = 0.80

must mean approximately:

80% observed event frequency

then calibration matters.

7. Training cost

A 0.1% improvement may not justify dramatically more tuning or infrastructure complexity.

8. Serving requirements

Suppose:

latency SLA = 2 ms

A small tree may be preferable despite lower predictive performance.

9. Interpretability requirement

If regulators/business users require transparent decision logic, model complexity could alter the selection.

10. Future regime changes

Most importantly:

A backtest validates performance under historical regimes, not all future regimes.

A new policy, recession, acquisition, vendor migration, accounting change, fraud strategy, or data pipeline can invalidate the historical conclusion.

54. How I would improve this experiment for production

Don't simply perform:

Train
Validation
Test

once.

Use rolling backtests.

For example:

Train 1-6 → validate 7-8

Train 1-8 → validate 9-10

Train 1-10 → validate 11-12

Train 1-12 → validate 13-14

...

Then compare:

mean performance
variance

worst period
recent performance
segment stability
calibration stability

A model with:

PR-AUC:
0.51
0.52
0.50
0.51

may be preferable to:

0.60
0.44
0.62
0.39

even if their averages are similar.

Stability itself has business value.

55. Senior-level model-selection framework

When someone asks:

| Why did you choose gradient boosting?

A strong reasoning structure is:

```
1. Business decision
  ↓
What prediction/action are we supporting?

2. Data modality
  ↓
Mostly structured/tabular

3. Baselines
  ↓
Linear/rules + decision tree

4. Candidate families
  ↓
RF + boosted trees

5. Validation design
  ↓
```

Point-in-time temporal backtesting

6. Metrics

↓

Business-cost aligned

7. Stability

↓

Period + geography + segment

8. Explainability

↓

Global + local

9. Operational constraints

↓

Latency / memory / cost

10. Production monitoring

↓

Performance + drift + calibration

That answer demonstrates much more maturity than:

"We used XGBoost because it usually performs well."

56. Day 10 mental model

Remember these three models this way:

DECISION TREE

"Find rules."

Risk:

memorizes data.

RANDOM FOREST

"Build many noisy rule systems
and average them."

Benefit:

variance reduction.

GRADIENT BOOSTING

"Build one rule system,
then repeatedly correct its mistakes."

Benefit:
powerful nonlinear tabular prediction.

And remember the three major boosting libraries this way:

XGBoost
→ strong mature general-purpose boosting

LightGBM
→ efficiency and large-scale training

CatBoost
→ categorical-heavy data

The most important senior-level conclusion for finance is:

Tree models are powerful because they easily learn thresholds and interactions. That same power makes them excellent at learning leakage, temporary operational artifacts, and historical policies. Therefore validation design, point-in-time correctness, temporal backtesting, and drift analysis are just as important as model choice.

For Day 10, if you can clearly explain **why a random forest reduces variance, why boosting fits residual/gradient errors sequentially, why learning rate interacts with tree count, why CatBoost is attractive for categoricals, and why a temporal test can overturn a model-selection conclusion**, you have the core tree-model depth expected at senior applied-ML level.

Day 10 DSA — Recursion

Beginner-friendly summary

Recursion is a technique where a function solves a problem by calling itself on a **smaller version of the same problem**.

Every correct recursive solution needs three things:

1. Base case
→ when does recursion stop?
2. Progress
→ does every call move toward the base case?
3. Recursive relation
→ how does the smaller solution help solve the current problem?

Typical uses include:

Trees / DFS
Backtracking
Binary search

Merge sort
Quick sort
Divide-and-conquer
Recursive mathematical relationships

1. Recognition signals

Consider recursion when you see these patterns:

Signal	Likely technique
Problem contains smaller versions of itself	Recursion
Tree or hierarchy	Recursive DFS
Split input into halves	Divide-and-conquer
Generate all combinations	Backtracking
Explore every path	DFS / recursion
Nested data structures	Recursive traversal
Repeated mathematical relationship	Recurrence

A useful question is:

If I already knew the answer to a smaller version of the problem, could I easily construct the current answer?

If yes, recursion may be appropriate.

2. The call stack

Consider:

```
def factorial(n: int) -> int:
    if n <= 1:
        return 1

    return n * factorial(n - 1)
```

For:

```
factorial(4)
```

calls happen like this:

```
factorial(4)
  |
factorial(3)
  |
factorial(2)
```

```
|  
factorial(1)  
|  
1
```

Then results return upward:

```
factorial(1) = 1  
  ↑  
factorial(2) = 2 × 1 = 2  
  ↑  
factorial(3) = 3 × 2 = 6  
  ↑  
factorial(4) = 4 × 6 = 24
```

Python keeps every unfinished function call on the **call stack**.

Conceptually:

```
Top  
-----  
factorial(1)  
factorial(2)  
factorial(3)  
factorial(4)  
-----  
Bottom
```

Each stack frame contains information such as:

- arguments
- local variables
- return location
- intermediate state

Therefore recursive depth contributes to **space complexity**.

3. Base cases

A base case tells recursion when to stop.

Wrong:

```
def solve(n):  
    return solve(n - 1)
```

Nothing stops the recursion.

Eventually Python raises:

RecursionError

Correct:

```
def solve(n):  
    if n == 0:  
        return 0  
  
    return solve(n - 1)
```

But merely having a base case isn't sufficient.

This is still wrong:

```
def solve(n):  
    if n == 0:  
        return 0  
  
    return solve(n + 1)
```

For solve(5):

5 → 6 → 7 → 8 → ...

The recursive call moves away from the base case.

So always verify:

```
Base case exists  
+  
Every call progresses toward it
```

4. Recursion versus iteration

Many recursive solutions can also be implemented with loops.

Recursive factorial:

```
def factorial(n: int) -> int:  
    if n <= 1:  
        return 1  
  
    return n * factorial(n - 1)
```

Iterative:

```
def factorial(n: int) -> int:  
    result = 1
```

```

for value in range(2, n + 1):
    result *= value

return result

```

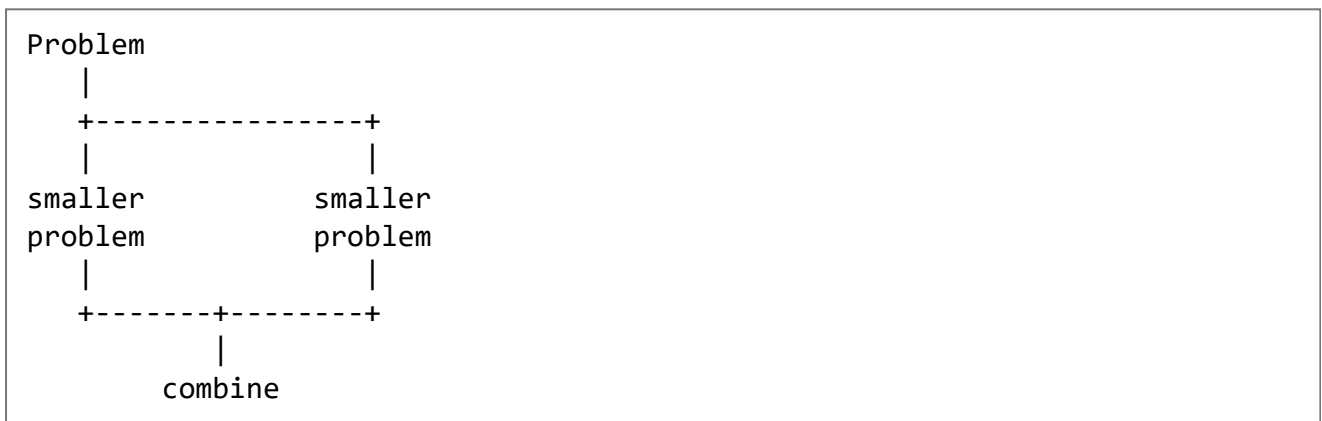
Comparison:

Approach	Time	Extra space
Recursive	$O(n)$	$O(n)$
Iterative	$O(n)$	$O(1)$

Python does not generally optimize tail recursion, so iteration is often preferable when recursion does not make the solution significantly clearer.

5. Divide-and-conquer

Divide-and-conquer means:



Examples:

- binary search
- merge sort
- quicksort
- fast exponentiation

Today we'll solve a medium problem using this pattern.

Medium Problem — Pow(x, n)

Implement a function that calculates:

$$x^n$$

Examples:

```

x = 2
n = 10

```


Output:
1024

And:

x = 2
n = -2

Output:
0.25

because:

$$2^{-2} = \frac{1}{2^2} = \frac{1}{4}$$

6. Recognition signals

The obvious calculation is:

$$x^n = x \times x \times x \dots$$

n times.

But notice:

$$x^{10} = x^5 \times x^5$$

Similarly:

$$x^8 = x^4 \times x^4$$

So instead of reducing:

n
↓
n - 1
↓
n - 2
↓
...

we can reduce:

n
↓
n / 2
↓
n / 4
↓
n / 8

↓
...

Whenever the input can be halved repeatedly, think:

$$O(\log n)$$

This is the main recognition signal.

7. Brute-force reasoning

The simplest solution multiplies x by itself n times.

```
def my_pow(x: float, n: int) -> float:
    if n < 0:
        x = 1 / x
        n = -n

    result = 1.0

    for _ in range(n):
        result *= x

    return result
```

For:

2^5

we do:

```
1
× 2 = 2
× 2 = 4
× 2 = 8
× 2 = 16
× 2 = 32
```

Complexity:

```
Time:  $O(|n|)$ 
Space:  $O(1)$ 
```

If:

$n = 1,000,000,000$

this means roughly one billion iterations.

We can do much better.

8. Optimized reasoning

For even n:

$$x^n = x^{n/2} \times x^{n/2}$$

For example:

$$2^{10} = 2^5 \times 2^5$$

For odd n:

$$\begin{aligned} x^n &= x^{\lfloor n/2 \rfloor} \\ &\quad \times \\ &\quad x^{\lfloor n/2 \rfloor} \\ &\quad \times x \end{aligned}$$

For example:

$$2^5 = 2^2 \times 2^2 \times 2$$

So:

```
half = power(x, n // 2)

if n even:
    result = half * half

if n odd:
    result = half * half * x
```

9. Important optimization trap

Don't write:

```
return power(x, n // 2) * power(x, n // 2)
```

because you're calculating the same result twice.

That produces:

```
      n
     / \
    n/2  n/2
   / \  / \
  ...
```

Instead:

```
half = power(x, n // 2)
```

```
return half * half
```

Now recursion follows one chain:

```
n
|
n/2
|
n/4
|
n/8
|
...
|
0
```

That's what gives us logarithmic time.

10. Handling negative powers

Remember:

$$x^{-n} = \frac{1}{x^n}$$

So we can transform:

```
if n < 0:
    x = 1 / x
    n = -n
```

Then the recursive helper only needs to solve non-negative exponents.

11. Pseudocode

```
FUNCTION my_pow(x, n):
```

```
    IF n < 0:
        x = 1 / x
        n = -n
```

```
    RETURN fast_power(x, n)
```

```
FUNCTION fast_power(x, n):
```

```
    IF n == 0:
        RETURN 1
```

```
    half = fast_power(x, floor(n / 2))
```

```
IF n is even:
    RETURN half * half

ELSE:
    RETURN half * half * x
```

12. Python recursive solution

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x = 1 / x
            n = -n

        def fast_power(exponent: int) -> float:
            if exponent == 0:
                return 1.0

            half = fast_power(exponent // 2)

            if exponent % 2 == 0:
                return half * half

            return half * half * x

        return fast_power(n)
```

13. Walkthrough

Calculate:

```
2^10
```

Recursive calls:

```
power(10)
  |
power(5)
  |
power(2)
  |
power(1)
  |
power(0)
```

Now unwind.

For power(0):

1

For power(1):

```
half = 1  
  
1 × 1 × 2  
= 2
```

For power(2):

```
half = 2  
  
2 × 2  
= 4
```

For power(5):

```
half = 4  
  
4 × 4 × 2  
= 32
```

For power(10):

```
half = 32  
  
32 × 32  
= 1024
```

Final answer:

1024

14. Why the solution is correct

Suppose the recursive call correctly computes:

$$half = x^{\lfloor n/2 \rfloor}$$

Case 1: even exponent

Let:

$$n = 2k$$

Then:

$$x^n = x^{2k} = x^k \times x^k$$

Therefore:

```
half * half
```

is correct.

Case 2: odd exponent

Let:

$$n = 2k + 1$$

Then:

$$x^n = x^{2k+1} = x^k \times x^k \times x$$

Therefore:

```
half * half * x
```

is correct.

The base case:

$$x^0 = 1$$

completes the recurrence.

15. Complexity

Every recursive call halves n:

```
n
n/2
n/4
n/8
...
1
```

Number of calls is approximately:

$$\log_2 n$$

Therefore:

Time

$$O(\log |n|)$$

Space

There are $O(\log n)$ call-stack frames:

$$O(\log |n|)$$

16. Edge cases

n = 0

$$5^0 = 1$$

Handled by the base case.

Negative exponent

$$2^{-3}$$

Convert:

$$2^{-3} = (1/2)^3$$

Negative base

$$(-2)^3 = -8$$

$$(-2)^4 = 16$$

Odd/even handling naturally produces the correct sign.

x = 1

$$1^n = 1$$

Works naturally.

Fractional base

$$0.5^3 = 0.125$$

Also works.

Huge exponent

This is where the optimization is most important.

For:

$$n \approx 1,000,000,000$$

brute force may require around one billion iterations.

Fast power requires only about:

$$\log_2(10^9) \approx 30$$

recursive levels.

17. Recursion-to-iteration conversion

The recursive solution has:

```
Time:  O(log n)
Space: O(log n)
```

We can eliminate the call stack and achieve:

```
Time:  O(log n)
Space: O(1)
```

This technique is called **binary exponentiation**.

The recurring operations are:

```
if exponent is odd:
    include current x in result

square x

halve exponent
```

18. Iterative reasoning

Suppose:

```
n = 13
```

In binary:

```
13 = 11012
```

which represents:

$$13 = 8 + 4 + 1$$

Therefore:

$$x^{13} = x^8 \times x^4 \times x$$

Repeated squaring generates:

```
x
x2
x4
x8
x16
...
```

We use only the powers corresponding to 1 bits in the exponent.

19. Iterative pseudocode

```
IF n < 0:
    x = 1 / x
    n = -n

result = 1

WHILE n > 0:

    IF n is odd:
        result = result * x

    x = x * x

    n = floor(n / 2)

RETURN result
```

20. Optimized iterative Python

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x = 1 / x
            n = -n

        result = 1.0

        while n > 0:
            if n % 2 == 1:
                result *= x

            x *= x
            n //= 2

        return result
```

Complexity:

```
Time:  $O(\log |n|)$   
Space:  $O(1)$ 
```

For Python, this is usually the implementation I'd prefer if recursion itself isn't part of the requirement.

21. Recursive vs iterative

Property	Recursive	Iterative
Time	$O(\log n)$	$O(\log n)$
Space	$O(\log n)$	$O(1)$
Call-stack usage	Yes	No
Easy recurrence explanation	Excellent	Moderate
Large-depth safety	Lower	Better
Typical Python implementation choice	Good for teaching	Often preferable

22. General recursion-to-iteration idea

A recursive function like:

```
solve(state)
  |
  solve(next_state)
```

can sometimes become:

```
state = initial_state

while not finished:
    process state
    state = next_state
```

For recursive DFS, the conversion is particularly important:

```
Recursive DFS
  ↓
Python call stack
```

becomes:

```
Iterative DFS
  ↓
Explicit stack/list
```

Example:

```
stack = [start]

while stack:
    node = stack.pop()

    for neighbor in node.neighbors:
        stack.append(neighbor)
```

So recursion and stacks are deeply related:

Recursion uses an implicit stack; iteration can often replace it with either loop state or an explicit stack.

23. Common recursion mistakes

Missing base case

```
def solve(n):
    return solve(n - 1)
```

Moving away from the base case

```
def solve(n):
    if n == 0:
        return

    solve(n + 1)
```

Duplicate recursive computation

Bad:

```
power(x, n // 2) * power(x, n // 2)
```

Better:

```
half = power(x, n // 2)
return half * half
```

Forgetting stack-space complexity

This:

```
recursive depth = n
```

means:

```
O(n) stack space
```

even if no explicit list or dictionary was created.

Using recursion unnecessarily

For straightforward linear traversal:

```
for item in items:  
    ...
```

is generally better than recursive processing of millions of elements.

24. Recursion problem-solving template

Before coding, answer these questions:

1. What exactly does my recursive function return?
2. What is the smallest problem?
3. What is the base case?
4. How does the recursive call make the problem smaller?
5. What result will recursion return to me?
6. How do I combine that result?
7. How deep can recursion become?
8. Am I repeating the same recursive computation?
9. Would memoization eliminate repeated work?
10. Can iteration remove unnecessary stack usage?

For today's problem, the recursive contract is simply:

```
fast_power(n)  
=  
x raised to exponent n
```

Once that definition is clear, the recurrence follows naturally.

Day 10 DSA takeaway

Remember recursion as:

Define the recursive function
↓
Identify base case
↓
Reduce problem size
↓
Trust smaller recursive result
↓
Combine results

For divide-and-conquer, look specifically for:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8$

because that often signals:

$$O(\log n)$$

For today's medium problem:

Solution	Time	Space
Repeated multiplication	$O(n)$	$O(1)$
Recursive fast power	$O(\log n)$	$O(\log n)$
Iterative binary exponentiation	$O(\log n)$	$O(1)$

The important skill is not memorizing $\text{Pow}(x, n)$. It is recognizing the **halving structure**, defining the recursive contract correctly, avoiding duplicate recursive work, accounting for the call stack, and knowing how to convert recursion into iteration.